P SMARTPARK

Smart Car Park System

Project Report — an IoT and cloud-based real-time parking management platform, covering system architecture, implementation, and current build status.

Institution
Sheffield Hallam University

Author
[Student name]

Repository
Smart-Car-Park-System

Document type
Technical project report

Module / course
[Module code]

Report date
4 August 2026

Table of Contents

1.	1. Introduction	Background, problem statement, aims & objectives
2.	2. System Overview	Scope, users, high-level description
3.	3. System Architecture	Four-layer architecture and data flow
4.	4. Technology Stack	Languages, frameworks and libraries
5.	5. Data Model & Database Design	Entities, fields and relationships
6.	6. Frontend Implementation	Marketing site, customer & admin portals
7.	7. Backend Implementation	Django REST Framework services
8.	8. IoT Layer	Raspberry Pi and sensor integration (planned)
9.	9. Security Considerations	Authentication, data handling, risks
10.	10. Testing & Quality Assurance	What has been verified so far
11.	11. Implementation Status & Limitations	Honest state of the build
12.	12. Future Work	
13.	13. Conclusion	
14.	Appendix A — Repository Structure	
15.	Appendix B — Demo Credentials & Environment Variables	

1. Introduction

1.1 Background

Urban parking is a persistent source of congestion, wasted fuel and driver frustration: studies of city-centre traffic consistently attribute a significant share of circulating vehicles to drivers searching for a free space. The Smart Car Park System addresses this by combining IoT occupancy sensing with a cloud backend and a web application, so that drivers can see live availability and reserve a space before they arrive, rather than discovering an occupied lot on site.

1.2 Problem Statement

Conventional car parks give drivers no visibility of availability until they are already inside the facility. Operators, in turn, have no digital record of occupancy, no way to take reservations, and no automated way to control entry/exit barriers. This project builds the software layer needed to solve both sides of that problem: a customer-facing booking experience, and an operator-facing management console.

1.3 Aims and Objectives

- Monitor real-time parking slot availability across multiple levels and zones.
- Allow customers to search, filter and reserve a specific bay in advance.
- Generate a scannable QR code per booking for barrier entry.
- Give operators a dashboard to manage slots, bookings, gates and customers.
- Lay a REST API contract that a future IoT/sensor layer can populate with live occupancy data.
- Keep the frontend fully demoable independent of backend availability, using a swappable API layer.

1.4 Report Structure

Sections 2–4 describe the system at increasing levels of technical detail. Section 5 documents the data model. Sections 6–8 walk through each implementation layer (frontend, backend, IoT). Sections 9–11 cover security, testing and an honest account of what is built versus planned. Sections 12–13 close with future work and conclusions.

2. System Overview

The Smart Car Park System is split into two user-facing roles and one physical/data layer:

CUSTOMER

Registers an account, searches live availability, reserves a bay for a chosen time window, and presents a QR code at the barrier. Can view, extend awareness of and cancel their own bookings from a personal dashboard.

OPERATOR / ADMIN

Manages the parking slot inventory (add, edit, retire, price, EV-charger flag), reviews and cancels bookings across all customers, opens/closes physical gates remotely, and reviews registered customers.

IOT / SENSOR LAYER

Ultrasonic/IR sensors on a Raspberry Pi detect vehicle presence per bay and report status to the backend, which updates slot availability in real time. (See §8 for current status.)

A public marketing site introduces the product and funnels visitors to registration; authenticated users are routed into the customer portal, and operator accounts are routed into the admin console. Role separation is enforced client-side by a protected-route guard and, on the real deployment, would be enforced again server-side by the Django authentication layer.

3. System Architecture

The system follows a four-layer architecture:

```
            Vehicle
               |
               v
  Parking Detection Sensor  (ultrasonic / IR)
               |
               v
      Raspberry Pi          (IoT Layer)
               |
               v
     REST API Request
               |
               v
Django Backend (Cloud)      (Backend Layer)
               |
               v
  Cloud Database            (Database Layer)
               |
               v
React Web Dashboard         (Frontend Layer)
(Customer Portal / Admin Console)
```

3.1 IoT Layer

Components: Raspberry Pi, ultrasonic or infrared occupancy sensor per bay.
Function: detect whether a bay is occupied and push status changes to the backend over HTTP as they occur.

3.2 Backend (Cloud) Layer

Technology: Django 5 with Django REST Framework.
Function: receive sensor readings, persist parking slot / booking / gate state, expose REST endpoints consumed by the frontend, and enforce authentication.

3.3 Database Layer

Stores parking slot inventory, sensor status/history, booking records and gate state. See §5 for the full schema.

3.4 Frontend Layer

A single React application serving three surfaces from one codebase: the public marketing site, the authenticated customer portal, and the authenticated admin console — described in full in §6.

4. Technology Stack

4.1 Frontend

Concern	Choice	Notes
UI library	React 19.2	Function components + hooks throughout
Language	TypeScript ~6.0	Strict mode, no implicit any
Build tool	Vite 8	Dev server + production bundling
Styling	Tailwind CSS v4	Utility-first, brand tokens via `@theme`
Routing	React Router 7	Nested routes, protected route guard
Icons	lucide-react	Tree-shakeable SVG icon set
QR codes	qrcode.react	Client-side, no third-party network call

4.2 Backend

Concern	Choice	Notes
Framework	Django 5.2.15	App name `back1`, project name `monolish`
API layer	Django REST Framework 3.17	Serializer + APIView pattern
CORS	django-cors-headers 4.9	Allows the React dev server to call the API
Database driver	mysqlclient 2.2	MySQL-compatible database target
Deployment	gunicorn 26 + Dockerfile	WSGI server, containerised
Config	python-dotenv	Secrets loaded from `backend/utility/.env`

4.3 IoT (planned)

Concern	Choice
Controller	Raspberry Pi
Sensing	Ultrasonic or infrared distance sensor per bay
Transport	HTTP POST to Django REST endpoint

5. Data Model & Database Design

The current Django schema (`backend/back1/models.py`) defines three entities. Fields are intentionally minimal at this stage; the frontend's TypeScript types (§6.2) extend them with presentation fields — level, zone, price, EV-charger flag — that the schema will need to grow into before the real API can fully replace the mock layer.

5.1 ParkingSlot

Field	Type	Notes
id	AutoField	Primary key
slot_number	CharField(10)	e.g. `A-01`
is_available	BooleanField	Default `True`; flipped by sensor readings or bookings

5.2 Booking

Field	Type	Notes
id	AutoField	Primary key
user_name	CharField(100)	Denormalised customer name
parking_slot	ForeignKey → ParkingSlot	`on_delete=CASCADE`
start_time / end_time	DateTimeField	Reservation window
qr_code	CharField(255)	Barrier-entry reference, e.g. `SP-BK-00001`
created_at	DateTimeField	Defaults to `timezone.now`

5.3 Gate

Field	Type	Notes
id	AutoField	Primary key
gate_name	CharField(50)	Default `"Entrance Gate"`
is_open	BooleanField	Default `False`; toggled by admin action

> **Design note**
> The relationship is 1-to-many: a `ParkingSlot` can have many historical `Booking` rows, but a slot's `is_available` flag reflects only its current state. Historical bookings are not deleted when a slot is later re-booked, so occupancy history is preserved for reporting.

5.4 Fields the frontend expects but the backend does not yet store

Entity	Field	Purpose
ParkingSlot	level, zone	Physical location, used for search filters and occupancy-by-level reporting
ParkingSlot	has_ev_charger	EV filter in the customer search
ParkingSlot	price_per_hour	Billing/cost calculation
Booking	status	upcoming / active / completed / cancelled — currently derived client-side from start/end time
Booking	total_cost	Computed at booking time from duration × rate
Gate	location	Human-readable gate location
User	role	customer / admin — Django's built-in auth has no role field yet

6. Frontend Implementation

The frontend is a single Vite + React + TypeScript application (`/frontend`) covering three surfaces from one codebase and one design system.

6.1 Marketing site

A public landing page (`Homepage.tsx`) modelled on a premium city-parking template: a full-bleed hero with a "Quick Services Request" enquiry form, a benefits grid ("Why Choose Us"), an app-download band, a three-step "How It Works" panel, an FAQ accordion, and a contact section — all funnelling into `/register` or `/login`.

6.2 API abstraction layer

All data access goes through a single typed interface (`src/api/contract.ts`) implemented twice:

- **Mock implementation** (`src/api/mock/`) — persists to `localStorage`, seeded with realistic demo data (12 slots across 3 levels, 2 demo accounts, sample bookings and gates), with simulated network latency and real business rules (a slot cannot be double-booked; cancelling a booking frees its slot).
- **HTTP implementation** (`src/api/http.ts`) — a `fetch`-based client against the documented REST contract, token-authenticated, ready to point at Django once the matching endpoints exist.

Which implementation is active is controlled by a single environment flag (`VITE_USE_MOCK`), so every component, page and hook is written once against the interface and is unaffected by the switch. This lets the whole application be demonstrated, marked or user-tested without a running backend.

6.3 Authentication

`AuthContext` restores a session from a stored token on load, exposes `login/register/logout`, and a `ProtectedRoute` component redirects unauthenticated visitors to `/login` and redirects a logged-in user of the wrong role to their own portal — e.g. a customer hitting `/admin` is bounced to `/customer`.

6.4 Customer portal (`/customer`)

Page	Purpose
Overview	Active-session card, upcoming/spend statistics, recent activity feed
Find Parking	Filterable grid of live bays (zone, EV-only), reservation modal with time window and live cost
My Bookings	Status-tabbed list (all/upcoming/active/completed/cancelled) with one-click cancel
Booking Detail	Full reservation record plus a scannable QR code for barrier entry
Profile	Edit name/email, change password

6.5 Admin console (`/admin`)

Page	Purpose
Overview	Occupancy %, active bookings, customer count, today's revenue; occupancy-by-level bars; recent bookings
Parking Slots	Full CRUD on bays, inline availability toggle, EV/price/zone fields
Bookings	Search and status-filter across every customer's bookings, cancel any booking
Gates	Remote open/close per physical gate
Customers	Registered customers with per-customer booking counts

6.6 Shared layout & design system

Both portals share one dashboard shell (`DashboardShell.tsx`): a dark sidebar with role-specific navigation, an account card and logout, and a responsive mobile drawer. Shared UI primitives (`StatCard`, `Badge`, `Modal`, `EmptyState`, `PageHeader`, `Spinner`) keep every page visually consistent. The brand palette — lime accent (`#c3e02b`) on near-black (`#101010`) — is defined once as Tailwind theme tokens and used across the marketing site, both portals, and this report.

7. Backend Implementation

The Django project (`monolish`) contains a single app, `back1`, with three models (§5), DRF serializers for each, and a mix of function-based API views and class-based template views.

7.1 What exists today

- Session-based login via Django's built-in `auth_views.LoginView` at `/login/`.
- A server-rendered dashboard view (`views.dashboard`) that lists all slots, bookings and gates via Django templates.
- DRF `ModelSerializers` for `ParkingSlot`, `Booking` and `Gate` (`serializers.py`).
- API view functions `parking_slots_api`, `bookings_api`, `gates_api` returning serialized querysets.
- Class-based views for sensor readings (list/detail/create), gated behind `LoginRequiredMixin` where appropriate.
- CORS enabled so a separately-hosted React frontend can call the API in development.

7.2 Known gaps against the frontend's contract

> **Not yet wired up**
>
> `urls.py` currently only routes the dashboard, sensor endpoints and the class-based reading views — the `parking_slots_api`, `bookings_api` and `gates_api` view functions exist but are not yet exposed on a URL path. There is also no registration API, no token-authentication endpoint, no `role` field on the user model, and no booking-cancel or gate-toggle action endpoint. Section 6.2's HTTP client documents the REST paths the backend should grow into (`/api/auth/*`, `/api/parking-slots/`, `/api/bookings/`, `/api/gates/`, `/api/dashboard/stats/`).

8. IoT Layer

The intended occupancy-sensing layer — a Raspberry Pi per site polling an ultrasonic or infrared sensor per bay and posting status changes to the Django API — is documented in the system architecture (§3.1) but not yet implemented in this repository; the `raspberry-pi/` directory currently contains only a placeholder README.

Recommended integration path once hardware is available:

1. Sensor loop on the Pi reads distance/IR state per bay on a fixed interval (e.g. every 2 seconds).
2. On a state change, the Pi POSTs `{ slot_number, is_available }` to a dedicated ingestion endpoint.
3. The backend updates the corresponding `ParkingSlot` row and, if a booking is currently active on that slot, cross-checks for a mismatch (e.g. an unbooked car occupying a reserved bay) to flag for operator review.
4. The existing sensor-reading models/views (`SensorReadingListView`, etc.) already provide a place to log the raw readings for history and diagnostics.

9. Security Considerations

9.1 Implemented / designed for

- Role-based routing: the frontend guard prevents a customer session from ever rendering admin views, and vice versa.
- Passwords are never stored on the client beyond the submission event; the mock layer stores them only in `localStorage` for demo purposes and is clearly documented as non-production.
- Token-based request authentication is built into the real HTTP client (`Authorization: Token …` header), ready for DRF's token or session auth.
- CORS is explicitly configured on the backend rather than left open by default.

9.2 Outstanding for a production deployment

- The Django `SECRET_KEY` currently falls back to a hard-coded development value if no environment variable is set — this must be overridden in any deployed environment.
- `ALLOWED_HOSTS = ["*"]` is a development convenience and should be restricted per environment.
- No password-hashing/verification logic exists yet in the mock layer's real-backend counterpart beyond what Django's auth system provides by default — real user passwords must go through Django's `PASSWORD_HASHERS`, never stored or compared in plaintext.
- No rate limiting or brute-force protection is yet configured on the login endpoint.
- QR-code booking references should be treated as bearer credentials at the barrier — a production system should pair them with a time-bound validity check server-side, not just visual scanning.

10. Testing & Quality Assurance

10.1 Verified

Check	Method	Result
Type safety	`tsc -b` across the whole frontend	Pass
Production build	`vite build`	Pass
Route resolution	Dev server smoke test of every top-level route (`/`, `/login`, `/register`, `/customer`, `/admin`)	Pass
Mock business logic	Manual walkthrough: booking a slot marks it unavailable; cancelling frees it; double-booking an occupied slot is rejected	Pass

10.2 Not yet covered

- No automated unit or integration test suite exists yet on either the frontend or backend (`backend/back1/tests.py` is currently empty).
- No end-to-end browser tests (e.g. Playwright/Cypress) covering the booking or admin-management flows.
- No load testing of the Django API for concurrent booking requests against the same slot.
- No accessibility audit (keyboard navigation, screen-reader labelling) has been performed, though interactive elements use semantic HTML and ARIA attributes where applicable (e.g. `aria-expanded` on the FAQ accordion, `aria-label` on icon-only buttons).

> **Recommended next testing steps**
>
> Add Django `APITestCase` coverage for each model's CRUD endpoints once wired up; add React Testing Library coverage for the booking and cancellation flows against the mock API (since it requires no backend, it is well suited to CI); add a Playwright smoke test that logs in as each demo role and asserts the correct portal renders.

11. Implementation Status & Limitations

Layer	Status	Detail
Marketing site	Complete	Fully built and styled
Customer portal	Complete	Fully functional against the mock API
Admin console	Complete	Fully functional against the mock API
Backend data models	Complete	ParkingSlot, Booking, Gate defined and migrated
Backend REST API	Partial	Serializers and view functions exist; not fully routed; no auth/registration endpoints yet
Frontend ↔ real backend integration	Planned	Frontend currently runs on an in-browser mock by design; switching is a one-line env change once the API is complete
IoT sensor layer	Planned	Architecture documented; no hardware code in this repository yet
Cloud deployment	Planned	Dockerfile present for the backend; no deployed environment documented
Automated tests	Planned	No test suite yet on either side

This project deliberately decoupled the frontend from backend readiness: rather than blocking UI development on IoT hardware and a finished API, the frontend was built against a strict typed contract with a fully-featured local mock standing in for the network. This is a common pattern in professional frontend engineering and means the user-facing product can be demonstrated, tested and iterated on independently of backend and hardware progress.

12. Future Work

1. **Complete the REST API** — route the existing `parking_slots_api/bookings_api/gates_api` views, add registration, token auth, cancel and gate-toggle actions, and a `role` field on the user model.
2. **Extend the data model** — add `level`, `zone`, `price_per_hour` and `has_ev_charger` to `ParkingSlot`; add computed `status/total_cost` to `Booking`; add `location` to `Gate`, matching §5.4.
3. **Build the Raspberry Pi client** — a Python polling loop reading a GPIO-connected ultrasonic sensor and posting occupancy changes to the new ingestion endpoint.
4. **Switch the frontend off the mock** — set `VITE_USE_MOCK=false` and `VITE_API_URL` once the above is live; no frontend code changes required.
5. **Payments** — integrate a real payment provider (e.g. Stripe) behind the existing "Quick Services Request" and booking-confirmation flows.
6. **Notifications** — SMS/email reminders before a session ends, referenced in the marketing FAQ copy but not yet built.
7. **Automated testing** — as detailed in §10.2.
8. **Cloud deployment** — containerise and deploy the Django backend (Dockerfile already exists) behind a managed database, and deploy the frontend as a static build.

13. Conclusion

The Smart Car Park System demonstrates a complete, professional-grade frontend for a real-time parking platform — a public marketing site, a customer booking portal with QR-code entry, and an operator admin console — built against a strict, typed API contract that keeps it independent of backend and IoT readiness. The Django backend has its core data model in place and a working authentication and template-dashboard layer, with a clear, documented path to completing its REST API surface. The IoT sensing layer is architected but not yet implemented in hardware or software. Taken together, the project shows a coherent end-to-end design even where individual layers remain at different stages of completion, and the remaining work is well scoped in §12.

Appendix A — Repository Structure

```
Smart-Car-Park-System/
├── backend/                Django project ("monolish") + app ("back1")
│   ├── back1/               models.py, views.py, serializers.py, urls.py
│   ├── monolish/             settings.py, urls.py (project-level)
│   ├── templates/            server-rendered dashboard templates
│   └── Dockerfile
├── frontend/                React + TypeScript + Vite application
│   └── src/
│       ├── api/               typed contract, mock backend, HTTP client
│       ├── components/        layout, ui primitives, marketing components
│       ├── context/           AuthContext
│       ├── pages/
│       │   ├── auth/           Login, Register
│       │   ├── customer/       Overview, FindParking, MyBookings, ...
│       │   └── admin/          Overview, Slots, Bookings, Gates, Customers
│       └── Homepage.tsx       public marketing site
├── raspberry-pi/            IoT client (placeholder)
├── cloud/                   deployment notes (placeholder)
├── documentation/           this report, architecture notes
└── testing/                  test-results notes (placeholder)
```

Appendix B — Demo Credentials & Environment Variables

B.1 Demo accounts (mock API, seeded automatically)

Role	Email	Password
Admin	admin@smartpark.io	admin123
Customer	jordan@example.com	customer123

B.2 Frontend environment variables (`frontend/.env.local`)

Variable	Default	Purpose
VITE_USE_MOCK	*unset* (= true)	Set to `false` to call the real Django API instead of the mock
VITE_API_URL	`http://localhost:8000`	Base URL for the real backend when mock mode is off

B.3 Backend environment (`backend/utility/.env`)

Variable	Purpose
SECRET_KEY	Django secret key — must be overridden outside local development
WEBSITE_NAME	Presence of this variable disables `DEBUG` mode
DEBUG	Explicit debug toggle